

Name

Saghir Ali

Sap-id

67005

Assignment

(3) (AI 4-1)

Date

4-15-26

(1)

Insertion Sort :-

Pseudocode:-

for $i = 1$ to $n-1$: $key = arr[i]$ $j = i-1$ while $j \geq 0$ and $arr[j] > key$: $arr[j+1] = arr[j]$ $j--$ $arr[j+1] = key$

Algorithm:-

Start

Taking array of n $i = 1$ Set $key = arr[i]$ $j = i-1$ if $arr[j] > key$ then shift $arr[j]$ to right

Repeat until

$j < 0$ or $arr[j] \leq key$

Increment i and repeat

$i = n$

Stop

C++ code:-

```
#include <iostream>
```

```
using namespace std;
```

```
void insertionSort (int arr[], int n)
```

```
{ for (int i=1; i<n; i++)
```

```
    int key = arr[i]; //element
```

```
    int j = i - 1;
```

```
    // Shift elements greater than key  
    one position ahead.
```

```
    while (j >= 0 && arr[j] > key) {
```

```
        arr[j+1] = arr[j]
```

```
        j--;
```

```
    }
```

```
    arr[j+1] = key // place key
```

```
int main() {
```

```
    int arr[] = {12, 11, 13, 5, 6}
```

```
    int n = 5;
```

```
    insertionSort (arr, n);
```

```
    for (int i = 0; i < n; i++)
```

```
        cout << arr[i] << " ";
```

```
    return 0;
```

```
}
```


DATE: ___/___/___

DAY: ___/___/___

Time Complexity:-

Best (sorted)	$O(n)$
Average	$O(n^2)$
Worst (reverse)	$O(n^2)$

(2)

Bubble Sort :-

Pseudocode :-

```
for i = 0 to n-1;
  for j = 0 to n-i-2;
    if arr[j] > arr[j+1]:
      swap (arr[j], arr[j+1]);
```

Algorithm :-

Start

Array of n elements

Set $i = 0$

Set $j = 0$

Compare $arr[j]$ with $arr[j+1]$

If $arr[j] > arr[j+1]$, swap

Increment j and repeat till

$n-i-2$

Increment i and repeat outer

loop.

If no swap in a pass

Stop.

DATE: ___/___/___

DAY: ___/___/___

C++ code:-

```
void bubbleSort (int arr[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        bool swapped = false;  
        // optimization flag  
        for (int j = 0; j < n - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap (arr[j], arr[j + 1]);  
                // swap if out of order  
                swapped = true;  
            }  
        }  
        if (!swapped) break;  
    }  
}
```

```
int main () {  
    int arr[] = {64, 34, 25, 12, 22};  
    int n = 5;  
    bubbleSort (arr, n);  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
    return 0;  
}
```

Time complexity:-

Best (sorted)	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

(3)

Selection Sort:

Pseudo code:-

```
for (i = 0 to n-1):  
    minIdx = i  
    for (j = i+1 to n):  
        if arr[j] < arr[minIdx]:  
            minIdx = j  
    Swap (arr[i], arr[minIdx])
```

Algorithm:-

Start

Take array of n

Set i = 0

set minIdx = i

set j = i + 1

compare arr[j] with arr[minIdx]

if (arr[j] < arr[minIdx])

update minIdx = j

increment j and repeat

swap arr[j] with arr[minIdx]

increment i and repeat

Stop

C++ code

```
void selectionSort (int arr[], int n){  
    for (int i = 0; i < n-1; i++)
```


DATE: ___/___/___

DAY: ___/___/___

```
{ int minIdx = i;
  // current is minimum
  int minIdx = i;
  // find actual minimum
  for (int j = i+1; j < n; j++) {
    if (arr[j] < arr[minIdx])
      minIdx = j;
  }
  swap (arr[i], arr[minIdx]);
}
```

```
int main() {
  int arr[] = {64, 25, 12, 22, 11};
  int n = 5;
  selectionSort (arr, n);
  for (int i = 0; i < n; i++)
    cout << arr[i] << " ";
  return 0;
}
```

Time complexity:-

Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$

(4)

Quick Sort :**Pseudo code :**

```

quickSort (arr, low, high):
    if low < high:
        pi = Partition (arr, low, high)
        quickSort (arr, low, pi-1)
        quickSort (arr, pi+1, high)

```

```

Partition (arr, low, high):

```

```

    pivot = arr [high]

```

```

    i = low - 1

```

```

    for j = low to high - 1:

```

```

        if arr [j] <= pivot:

```

```

            i++

```

```

        swap arr [i], arr [j]

```

```

        swap (arr [i+1], arr [high])

```

```

    return i+1

```

Algorithm:

Start

Take array with low and high

if low $\geq$ high, return

Set pivot = arr [high]

Set i = low - 1

Set j = low

Compare arr [j] with pivot

if $arr[j] \leq \text{Pivot}$ then increment
 i and swap $arr[i]$, $arr[j]$

increment j , repeat:
swap $arr[i+1]$, $arr[\text{high}]$

Recursively call quicksort on
left (low to $Pi-1$)
or right ($Pi+1$ to high)

C++ Code:-

```
int Partition (int arr[], int  
low, int high) {
```

```
    int Pivot = arr[high]  
    // take last element  
    int i = low-1;
```

```
    for (j = low; j < high; j++) {  
        if (arr[j] <= Pivot) {  
            i++;
```

```
            swap (arr[i] , arr[j]);  
            // move small to left
```

```
        }  
    }
```

```
void quicksort (int arr[], int  
low, int high) {
```

```
    if (low < high) {
```



```

if (low < high) {
    int pi = Partition
        (arr, low, high)
    // left Part
    quickSort (arr, pi+1, high);
}
}

int main () {
    int arr [] = {10, 7, 8, 9, 1, 5};
    int n = 6;
    quickSort (arr, 0, n-1);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

Time complexity

Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n^2)$

Merge Sort: (5)

Pseudocode:

```

mergeSort (arr, left, right);

```



```

if left < right :
    mid = (left + right) / 2
    mergeSort(arr, left, mid)
    mergeSort(arr, mid+1, right)
    merge(arr, left, mid, right)

```

Algorithm:

```

Start
Take array left & right
if left >= right then
    return
Find mid (left + right) / 2
call merge sort to left
call merge sort to right
Create temp arrays L & R
Copy left half into L & R
set i = 0, j = 0, k = left
Compare L[i] with R[j]
Place smaller into arr[k]
increment pointer
Stop

```

C++ Code

```

void merge (int arr[], int left,
            int mid, int right) {
    int n1 = mid - left;
    int n2 = right - mid;
    int L[n1], R[n2],

```



```

// copy data to temp
for (int i = 0; i < n1; i++)
    L[i] = arr[left + i];
for (int j = 0; j < n2; j++)
    R[j] = arr[mid + 1 + j];

```

```

int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) arr[k++] = L[i++];
    else arr[k++] = R[j++];
}

```

```

while (i < n1) arr[k++] = L[i++];
while (j < n2) arr[k++] = R[j++];
}

```

```

void mergeSort (int arr[], int left,
                int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort (arr, left, mid);
        mergeSort (arr, mid + 1, right);
        merge (arr, left, mid, right);
    }
}

```

```

int main () {
    int arr[] = {38, 27, 43, 3, 9, 82};
    int n = 6;
    mergeSort (arr, 0, n - 1);
    for (int i = 0; i < n; i++)

```


DATE: ___/___/___

DAY: ___/___/___

```

    cout << arr[1] << " ";
    return 0;
}

```

Time complexity:-

Best	$O(n \log n)$
Average	$O(n \log n)$
Worst	$O(n \log n)$

(b)

Comparison:-

Algo	Best	Average	worst	Space	Stable
insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes ✓
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes ✓
selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No ✗
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No ✗
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes ✓